

Fantasy Baseball Dashboard Project: Code Improvement Request

1. Project Overview

I am developing a Streamlit application for managing my ESPN fantasy baseball team.

- * **League Type:** 12-team, Head-to-Head (H2H)
- * **Scoring Categories (10):**
 - * **Hitting:** Runs Scored (R), Home Runs (HR), Runs Batted In (RBI), Stolen Bases (SB), Batting Average (AVG)
 - * **Pitching:** Strikeouts (K), Wins (W), Earned Run Average (ERA), Walks Plus Hits Per Innings Pitched (WHIP), Saves Plus Holds (SVHD)
- * **League Home Page Example:**
`https://fantasy.espn.com/baseball/team?leagueId=64175&teamId=1&seasonId=2025`

Application Functionality:

- * **Data Source:** ESPN API (teams, rosters, past results).
- * **Weekly Analysis:** Dedicated Streamlit pages to plot weekly results for all teams across all categories.
 - * **Comparison:** Ability to select categories and teams for comparison.
 - * **Plot Values:** Weekly results, team average, league average.
- * **Player Rankings Integration:** Display rankings from various fantasy baseball sites.
 - * **Context:** Show player's fantasy team (or 'available' if unowned) based on ESPN roster data.

Current Status: The uploaded file `formatted_python_files` contains the python files for the program, separated by file name and location in the program folder.

2. Coding Guidelines for "Good Code"

When reviewing or generating code, adhere strictly to the following principles:

1. **Python Style (PEP 8):**
 - * 4-space indentation.

- * Limit line length.
 - * Effective blank line usage.
 - * Organized imports.
 - * Clear naming conventions (``snake\_case`` for functions/variables, ``CapWords`` for classes, ``ALL\_CAPS`` for constants).
 - * Prioritize readability.
2. **Logical Code Structure:**
 - * Use functions, modules, classes.
 - * Break down problems into smaller pieces.
 - * Define clear entry point (``if \_\_name\_\_ == "\_\_main\_\_":``).
 3. **Appropriate Data Structures:**
 - * Utilize built-in types (lists, dicts, tuples, sets) effectively.
 - * Choose simplest structure for clarity/maintainability.
 - * Consider ``enum`` or ``dataclasses`` where suitable.
 4. **Explicit Dependency Handling:**
 - * Declare external library dependencies (e.g., via PEP 723 comments).
 5. **Robust Error Handling & Feedback:**
 - * Use ``logging`` module (instead of ``print()``) for status/debugging.
 - * Graceful ``try...except`` error handling (specific exceptions).
 - * Use ``assert`` for internal consistency checks (development aid).
 - * (Optional) Enhance terminal output with libraries like ``Rich``.
 6. **Testable Code Design:**
 - * Design for easy testing from the outset.
 - * Consider testability even if full testing suite is not immediate.
 7. **Code Efficiency Consideration:**
 - * Mindful of performance while maintaining readability.
 - * Leverage Python optimized features (list comprehensions, generators, built-ins).
 - * Avoid unnecessary memory allocation or inefficient loops.
 8. **Object-Oriented Design (if applicable):**
 - * Apply principles like Single Responsibility Principle (SRP) for classes.

3. My Request

I require assistance in **improving my existing Python code** for this fantasy baseball dashboard.

****Action Required:****

Please start with the file `"""Fantasy Results.py"""` found in the uploaded pdf file. Suggest how this file can be improved following the guidelines I provided.